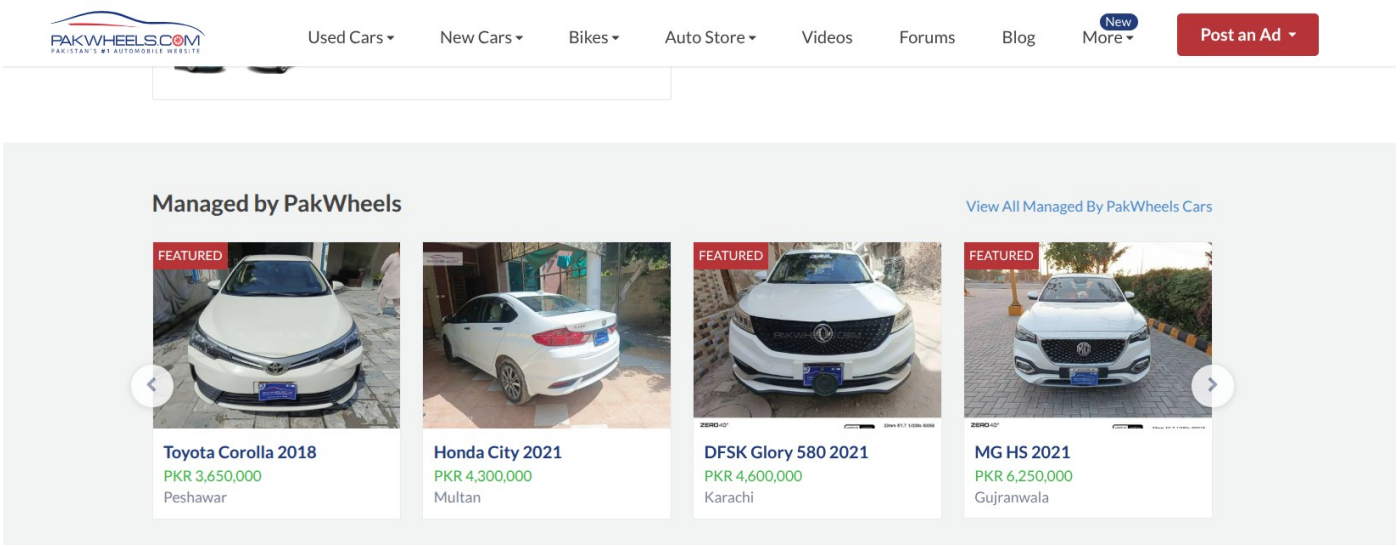
Documentation

Listing Class:



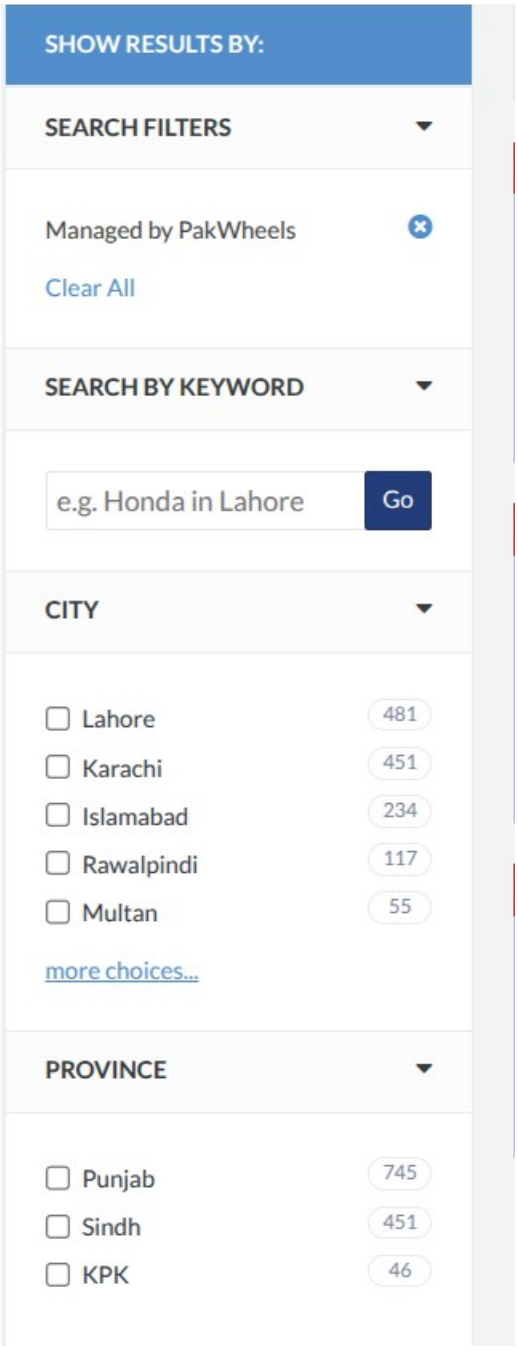

```cpp
class Listing{  //default needed for listings[] array, parameterized for actual listings
private:
    string listingID;
    Vehicle* vehiclePtr; //aggregation
    Seller* const sellerPtr;     //aggregation
    string status;     //cd be "Pending","Approved","Sold"
    string postDate;
    int price;
public:
    friend class Admin;
    Listing():vehiclePtr(nullptr), sellerPtr(nullptr), price(0) {}
    Listing(string id,Vehicle* v,Seller* s,string st,string d,int p):listingID(id),vehiclePtr(v),sellerPtr(s),status(st),postDa
    string getStatus() const { return status; }
    string getListingID() const { return listingID; }
    Vehicle* getVehicle() const { return vehiclePtr; }
    void setStatus(string s) { status = s; }
    void setPrice(int p) { price = p; }
    Listing& operator=(const Listing& other) {
    if(this != &other) {
        listingID = other.listingID;
        vehiclePtr = other.vehiclePtr;
        status = other.status;
        postDate = other.postDate;
        price = other.price;
```

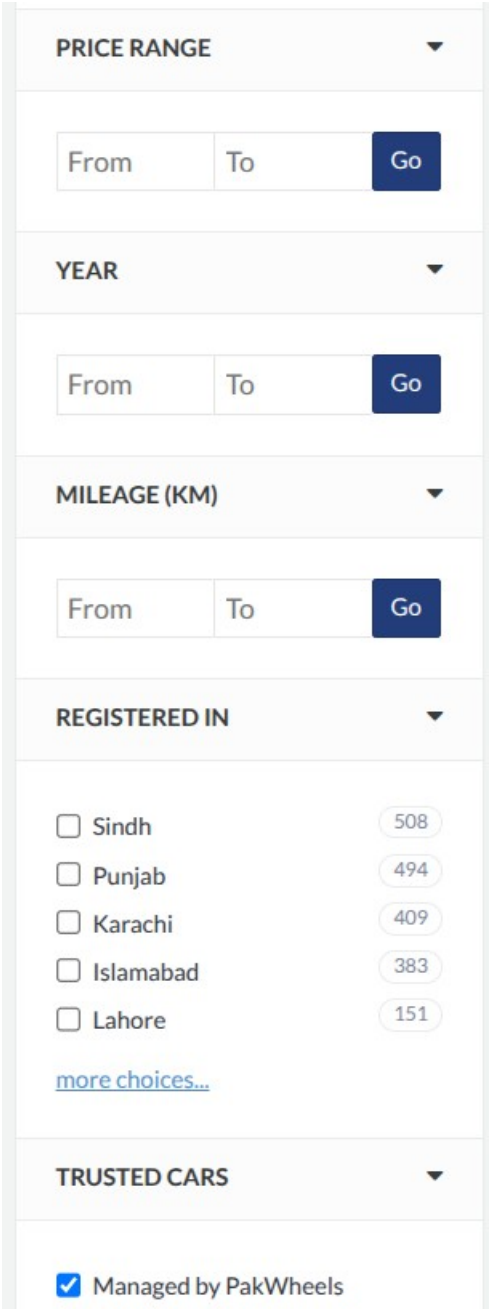
Reasoning:

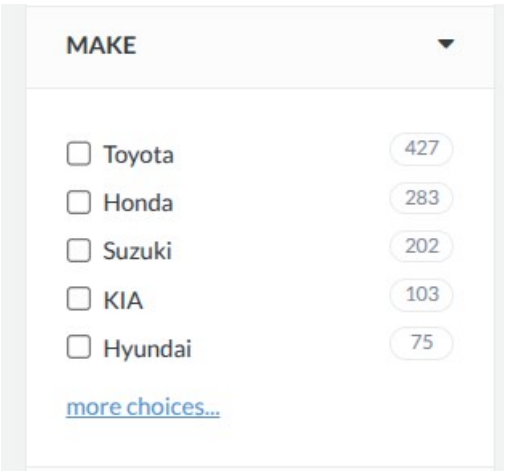
1. Vehicle Listings Page :(Listing Class)

PakWheels displays each car as a listing with price, status, and vehicle details. I replicated this with the **Listing Class** which stores a Vehicle* pointer, seller reference, price, status ("Pending" / "Approved" / "Sold") , and post date. Aggregation is used because the listing references a vehicle and seller without owning them because they exist independently.
Also Vehicle pointers are used to call overridden display() functions.
At runtime, the correct function (Car or Bike) is executed.
This demonstrates runtime polymorphism.

Search Filters:







```
    cout << "\nSearch by Brand: Toyota \n";
    for(int i=0;i<listingCount;i++){
        if(listings[i].getVehicle()->getBrand()=="Toyota")
            listings[i].showListing();
    }
    cout << "\nFilter by Model: Civic\n";
    for(int i=0;i<listingCount;i++){
        if(listings[i].getVehicle()->getModel()=="Civic")
            listings[i].showListing();
    }
    cout << "\nFilter by Price < 6,000,000\n";
    for(int i=0;i<listingCount;i++){
        if(listings[i].getVehicle()->getPrice()<6000000)
            listings[i].showListing();
    }
    cout << "\nFilter by Year >= 2021\n";
    for(int i=0;i<listingCount;i++){
        if(listings[i].getVehicle()->getYear()>=2021)
            listings[i].showListing();
    }
    cout << "\nFilter by Mileage < 30000\n";
    for(int i=0;i<listingCount;i++){
        if(listings[i].getVehicle()->getMileage()<30000)
            listings[i].showListing();
    }
```

Reasoning:

2. Search/Filter Options : filters in main() , PakWheels allows users to filter cars by brand, model, price, year, and mileage. I replicated this by iterating through listings and comparing getBrand( ), getModel( ), getPrice( ), getYear( ), and getMileage( ).The isAffordable(int) and isAffordable(int , int) overloaded functions demonstrate function overloading to support both single budget and price range filtering.

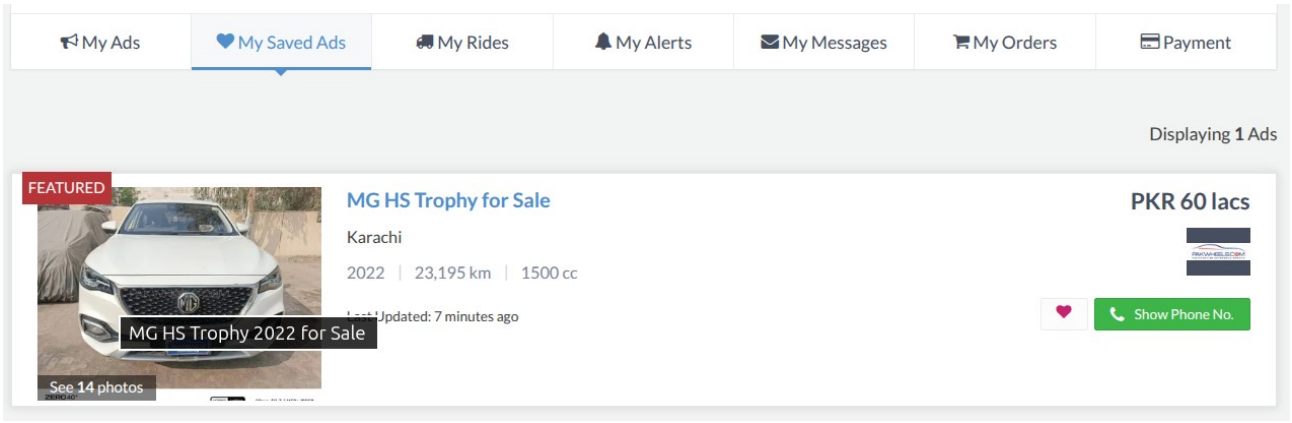
Seller Class:

Reasoning:

3. Seller Profile: Seller class PakWheels shows each seller with their name, contact, location, and rating. I replicated this by inheriting Seller from User (getting name, contact, address) and adding Ratings and a Vehicle* array for their listed cars. Inheritance is justified because a Seller IS-A User with additional selling-specific responsibilities.

```
class Seller:public User{ // default constructor for array/pointer use, parameterized for i
private:
    Vehicle* cars[10]; //seller can have up to 10 cars(composition)
    int carCount;
    string ratings;
public:
    Seller():carCount(0) {}
    Seller(string n,string ci,string a,string r): User(n,ci,a), ratings(r), carCount(0) {}
    string getRatings() const{ return ratings; }
    int getCarCount() const{ return carCount; }
    void setRatings(string r) { ratings = r; }
    bool hasAvailableCars() const { return carCount > 0; }
    void addCar(Vehicle* v) {
        if(carCount < 10) {
            cars[carCount++]= v;
            cout <<"Car added to seller "<< name << endl;
        }
    }
    void removeCar(int id) {
    for (int i = 0; i < carCount; i++) {
        if (cars[i]->getID() == id) {
            for (int j = i; j < carCount - 1; j++)
                cars[j] = cars[j + 1];
            carCount--;
            cout << "Car removed from seller " << name << endl;
            break;
        }
    }
}
    void display() const override {
    cout << "Seller: " << name << " Rating: " << ratings << "\n";}
};
```

Favourites Class:

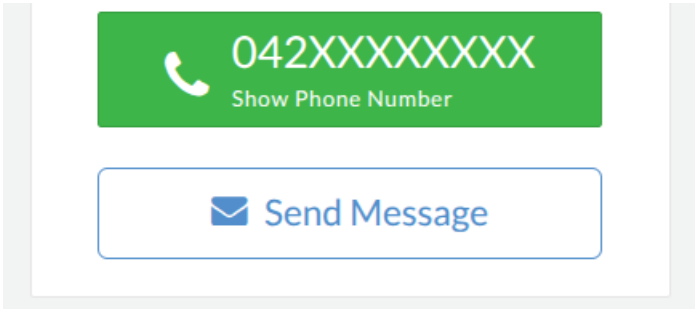



```
class Favorites{ //default ctor needed since Buyer has it as a member, parameterized if created alone
private:
    string buyerName;
    Vehicle* favoriteCars[10];
    int favoriteCount;
    const int maxFavorites;
    string dateOfLastUpdate;
public:
    Favorites():maxFavorites(10),favoriteCount(0){}
    Favorites(string bName,string date):buyerName(bName),dateOfLastUpdate(date),favoriteCount(0),maxFavorites(10){}
    int getFavoriteCount()const {return favoriteCount; }
    void addFavorite(Vehicle* v) {
        if(favoriteCount < maxFavorites) {
            favoriteCars[favoriteCount++]=v;
            cout <<"Saved: "<< v->getBrand()<<" " << v->getModel()<< "\n";
        }
    }
    void removeFavorite(int id) {
    for (int i = 0; i < favoriteCount; i++) {
        if (favoriteCars[i]->getID() == id) {
            for (int j = i; j < favoriteCount - 1; j++)
                favoriteCars[j] = favoriteCars[j + 1];
            favoriteCount--;
            return;
        }
    }
    }
    void showFavorites() const {
    for (int i = 0; i < favoriteCount; i++)
        favoriteCars[i]->display();
    }
    void clearFavorites(){ favoriteCount = 0; }
    friend void showBuyerFavorites(const Buyer& b, const Favorites& f);
};
```

Reasoning:

4. Favorites/Saved Cars : (Favourites class +Buyer) PakWheels lets buyers save cars to a favorites list for later viewing. I replicated this with the Favourites class composed inside Buyer , this is composition because favorites cannot exist without a buyer. It stores Vehicle* pointers and supports add, remove, and display operations. The friend function showBuyerFavourites( ) accesses private favorite data directly, justified because it needs cross-class access to both Buyer and Favourites private members.

Message Class:

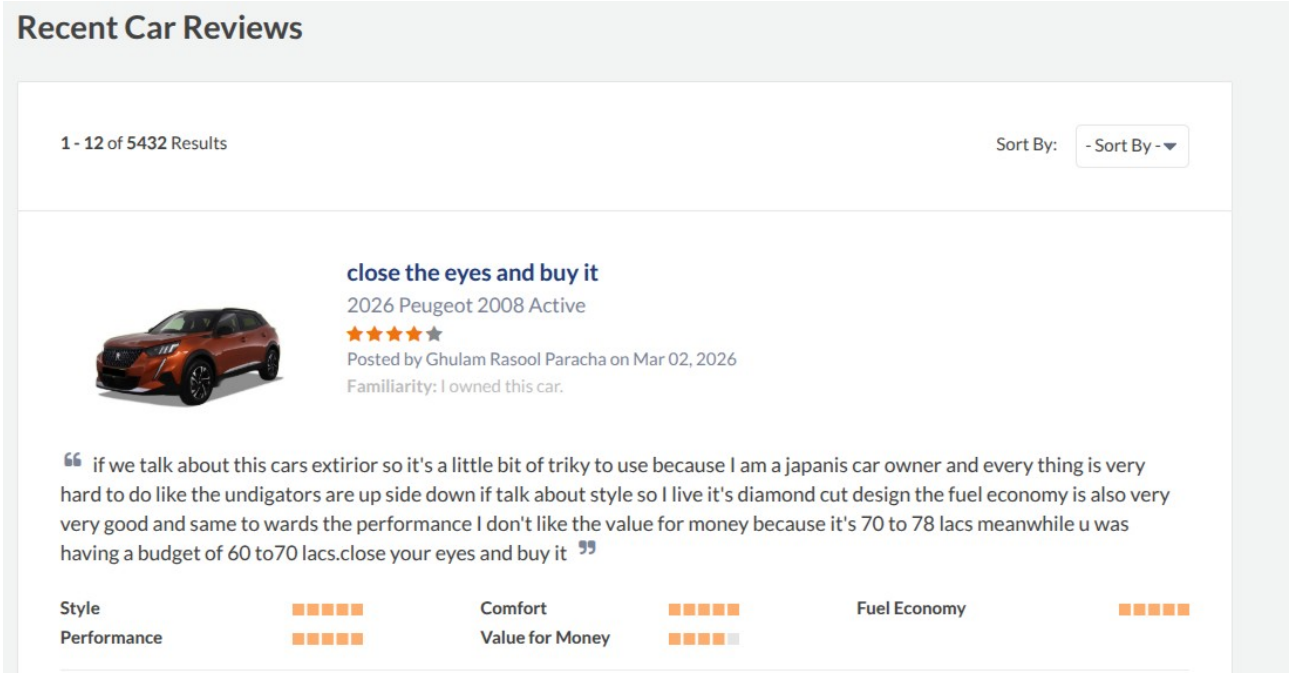



```
class Message{ //  default needed for messages[20] array, parameterized for actual messages
private:
    Buyer* senderPtr;   //aggregation
    Seller* receiverPtr;  //aggregation
    string content;
    string timestamp;
    bool readStatus;
public:
    Message(){}
    Message(Buyer* s,Seller* r,string c,string t,bool rs):senderPtr(s),receiverPtr(r),content(c),timestamp(t),readStatus(rs){}
    bool isRead() const { return readStatus; }
    void markAsRead() { readStatus = true; }
    void display() const { cout <<"Message: " << content <<" | Read: " << (readStatus ? "Yes" : "No") << "\n"; }
    void updateContent(string c){content = c; }
};
```

Reasoning:

5. Messaging System : Buyer:: sendMessage( ) PakWheels has a messaging feature where buyers contact sellers about listings.I replicated this with sendMessage(string) and sendMessage(string, time) which are two overloaded versions demonstrating function overloading. The Message class uses Buyer* and Seller* pointers (aggregation) because messages reference users without owning them.

Review/Ratings Class:



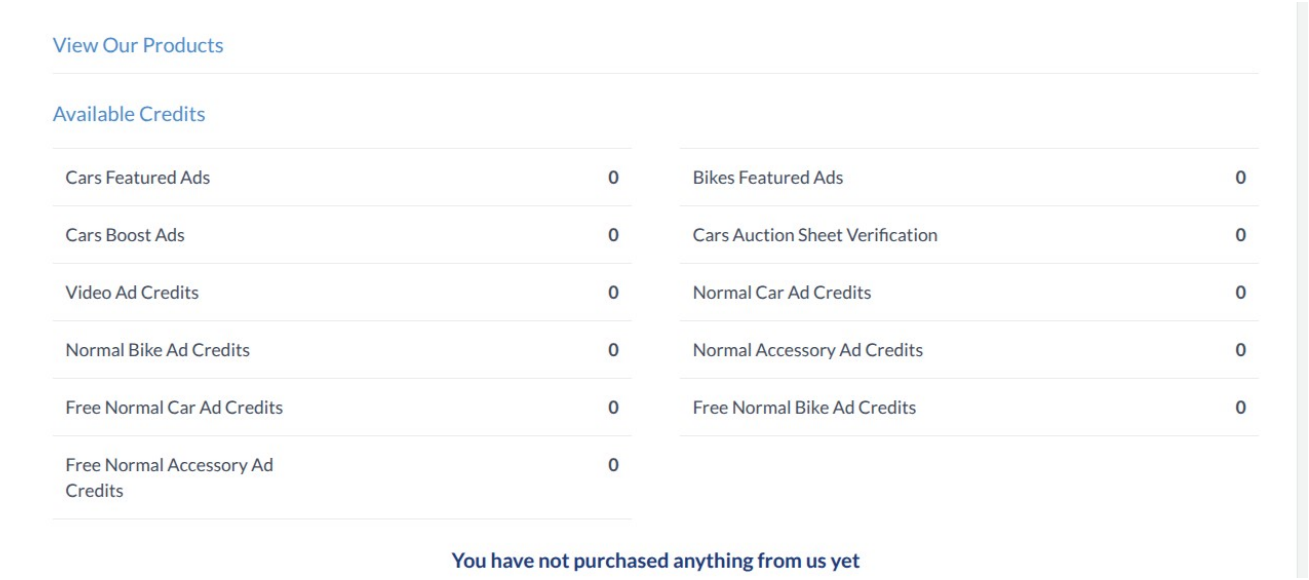

```cpp
class Reviews:public ReviewBase{  //default for array use, parameterized for creating reviews
private:
    string reviewerName;
    string reviewText;
    int ratingScore; // 1 to 5
    int relatedVehicleID;
    string relatedSellerName;
public:
    Reviews(){}
    Reviews(string rn,string rt,int rs,int vid,string sn):reviewerName(rn),reviewText(rt),ratingScore(rs),relatedVehicleID(vid),relatedSellerName(sn){}

    string getReviewerName() const { return reviewerName; }
    int getRatingScore() const { return ratingScore; }
    string getReviewText() const { return reviewText; }
    void setReviewText(string text) { reviewText = text; }
    void display() const { cout << reviewerName << "rated " << ratingScore << "/5: " << reviewText << "\n"; }
    bool isPositive() const { return ratingScore >= 4; }
    void updateRating(int r) { ratingScore = r; }
};
```

Reasoning:

6. Reviews/Ratings : Reviewes class PakWheels displays buyer reviews and star ratings for sellers and vehicles. I replicated this with the Reviiews class storing reviewerName, reviewText, ratingScore(1-5) , and the related vehicle and seller. The isPositive() function mirrors PakWheels' positive/negative review filtering, and updateRating( ) allows rating modification.

Payments class:



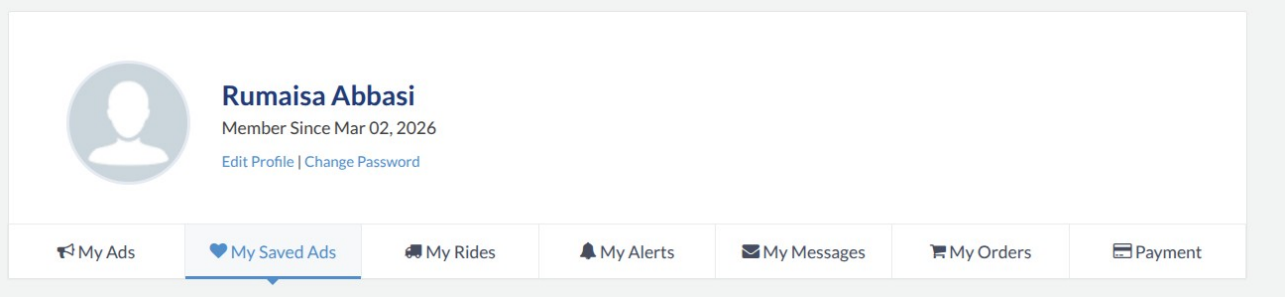

```
class Payments:public Payment{  //default for array use, parameterized for creating payments
private:
    Listing* listingPtr;
    string buyerName;
    string sellerName;
    string vehicleID;
public:
    Payments(){}
    Payments(string pid,Listing* l,string bn,string sn,string vid,int amt)
    : Payment(pid,amt), listingPtr(l),
      buyerName(bn), sellerName(sn), vehicleID(vid) {}
    string getPaymentID() const { return paymentID; }
    int getAmount() const { return amount; }
    string getBuyerName()const { return buyerName; }
    string getSellerName() const { return sellerName; }
    void display() const { cout <<"Payment " <<paymentID << " Amount: " << amount << "\n"; }
    void processPayment() const override {
    cout << "Processing payment of Rs." << amount << "\n";
}
    bool isValid()const { return amount > 0; }
    void updateAmount(int a) { amount = a; }
    bool operator==(const Payments& p) const {
    return this->paymentID == p.paymentID;}
    Payments operator+(int extra) const {
    Payments temp = *this;
    temp.amount += extra;
    return temp;}
    friend void showPaymentDetails(const Payments& p);
};
```

Reasoning:

7. Payments/Transactions : Payments class PakWheels facilitates payments between buyers and sellers for vehicle purchases. I replicated this by inheriting Payments from the abstract Payment class, which defines the pure virtual ProcessPayment( ) function. This demonstrates abstraction ; the interface is defined in Payment.h and the implementation is in Payments class . The friend function

showPaymentDetails( ) accesses private paymentIDand amount directly, justified because payment details need to be displayed without exposing public setters.

Buyer class:

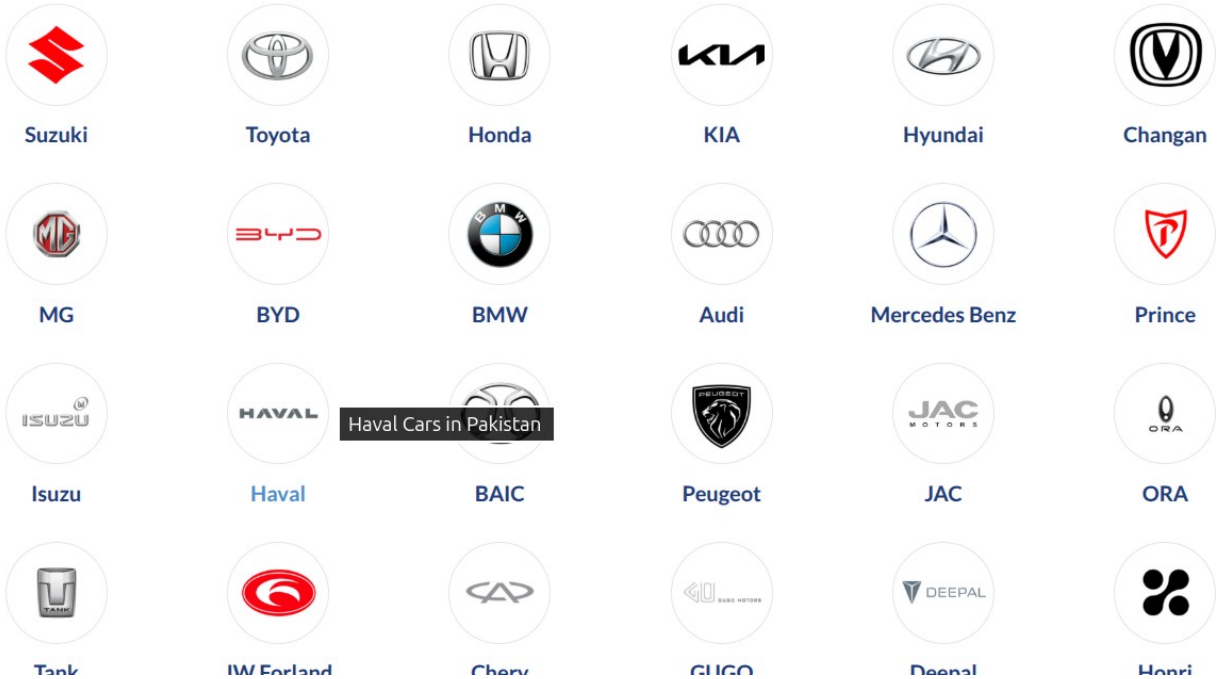



```
class Buyer:public User{ //default for array use, parameterized for creating buyers
private:
    Favorites myFavorites;   //composition
    Message messages[20];  //messages from sellers (composition)
    int messageCount;
public:
    Buyer():messageCount(0) {}
    Buyer(string n,string ci,string a): User(n,ci,a), messageCount(0){}
    int getMessageCount()const { return messageCount; }
    void saveFavorite(Vehicle* v) { myFavorites.addFavorite(v); }
    void showFavorites()const { myFavorites.showFavorites(); }
    void sendMessage(string msg) {
        cout << name << " sent: " << msg << "\n"; }
    void sendMessage(string msg, string time) {
        cout << name << " sent at " << time << ": " << msg << "\n"; }
    void showMessages()const { cout << name << "'s messages shown.\n"; }
    void receiveMessage(const Message& m) {
        if(messageCount < 20) {
            messages[messageCount++]=m;
        }
    }
    void deleteMessage(int index) {
        if(index >=0 && index < messageCount) {
            for(int i=index;i<messageCount-1;i++)
                messages[i]=messages[i+1];
            messageCount--;
        }
    }
    const Favorites& getFavorites() const { return myFavorites; }
    void display() const override {
    cout << "Buyer: " << name << endl;}
    friend void showBuyerFavorites(const Buyer& b, const Favorites& f);
};
```

Reasoning:

8. Buyer Account: buywr class PakWheels has registered buyer accounts with saved favorites, message history, and personal contact info. I replicated this by inheriting Buyer from `User` (getting name, contact, address) and adding a composed Favourites object and a Message array. Composition is used for both because favorites and messages belong exclusively to that buyer and cannot exist independently.

Brand class:




```
class Brand{  //default for array use, parameterized for initialization
private:
    string brandName;
    string country;
    const int yearFounded;
    string popularModels[5];
    int numberOfCarsListed;
public:
    Brand():yearFounded(0), numberOfCarsListed(0){}
    Brand(string b,string c,int y,int num):brandName(b),country(c),yearFounded(y),numberOfCarsListed(num){}
    string getBrandName() const{ return brandName; }
    string getCountry() const{ return country; }
    void display() const{ cout<<brandName << " from " <<country << " founded in " <<yearFounded <<"\n"; }
    void incrementListed() { numberOfCarsListed++; }
    void decrementListed() { numberOfCarsListed--; }
    bool isPopular() const { return numberOfCarsListed > 5; }
    void addPopularModel(string model) {
        for(int i=0;i<5;i++) {
            if(popularModels[i] == "") {
                popularModels[i] = model;
                break;
            }
        }
    }
    void showPopularModels()const {
        cout << "Popular models of " << brandName << ":\n";
        for(int i=0;i<5;i++)
            if(popularModels[i] != "")
                cout << popularModels[i] << endl;
    }
};
```

Reasoning:

9. Car Brands Page: Brand class PakWheels has a dedicated brands section showing each brand's country of origin, popular models, and number of listings. We replicated this with the Brand class storing brandName, country,yearFounded, (const since it never changes), a popularModels array, and numberOfCarslisted .
The isPopular( ) function mirrors PakWheels' featured brands logic, and incrementListed() / decrementListed( ) track active listings per brand.

Polymorphism is implemented using both function overriding and function overloading.

Function overriding is demonstrated in the Vehicle hierarchy where Car and Bike override the display() function. A Vehicle pointer is used to call display(), ensuring runtime polymorphism.

Function overloading is implemented in methods like isAffordable() which accepts different parameter lists, allowing flexible filtering of vehicles.

Abstraction is implemented by introducing abstract base classes such as User, Vehicle, and Payment.

These classes define only essential interfaces using pure virtual functions like display() and processPayment(), while hiding implementation details.

Derived classes such as Buyer, Seller, Admin, Car, Bike, and Payments implement these functions, ensuring a clear separation between interface and implementation.

Header files (.h) are used to define abstract classes, improving modularity, readability, and maintainability of the system.

Operator overloading is implemented to enhance usability and readability of the system.

The == operator is used to compare objects such as Vehicle, Listing, and Payments based on unique identifiers.

The + operator is used to combine or modify values such as price and payment amount.

The << and >> operators are overloaded to allow easy input and output of objects, making the system more user-friendly.

Both member and global operator functions are used to demonstrate flexibility.

Friend functions and friend classes are used to allow controlled access to private data where necessary.

The << and >> operators are declared as friend functions in the Vehicle class to allow direct access to private attributes for input/output operations.

A friend function is used between Buyer and Favorites to display a buyer's favorite vehicles, demonstrating interaction between two classes.

The Admin class is declared as a friend of Listing, allowing it to directly modify private attributes such as status and listingID. This is useful for administrative control over listings.

Friend functions improve flexibility while still maintaining encapsulation.